System messages are a specialized type of input that provides instructions, defines the AI's persona, or sets boundaries for its responses within the conversation. For instance, a system message might be "Act as a helpful and encouraging mentor." System messages provide persistent context for the AI's evaluation, whereas the main prompt is the user's direct query for a specific task.

One such library is *tiktoken*, originally developed for OpenAI's models but also useful for other providers with similar tokenization schemes. Let's explore how to use js-tiktoken, the JavaScript port of tiktoken, to calculate tokens for a given prompt in the following listing.

Listing 5.1 Function that counts prompt tokens

```
import { encoding_for_model } from '@dqbd/tiktoken';

function countTokens(text, model = "gpt-3.5-turbo") {
  const enc = encoding_for_model(model);
  const tokens = enc.encode(text);
  enc.free();
  return tokens.length;
}
const prompt =
  "Translate the following English text to French: 'Hello, how are you?'";
const systemMessage
  "You are a helpful assistant that translates English to French.";

const promptTokens = countTokens(prompt);
const systemTokens = countTokens(systemMessage);
const totalTokens = promptTokens + systemTokens;

console.log(`Prompt tokens: ${promptTokens}`);
console.log(`System message tokens: ${systemTokens}`);
console.log(`Total tokens: ${totalTokens}`);
```

Imports the encoding_for_model function from the tiktoken library. This function provides the appropriate tokenizer for different GPT models.

Encodes the input text into tokens. This step converts the text into a sequence of token IDs that the model understands.

Counts tokens for the main prompt. This helps ensure the prompt fits within token limits and aids in cost estimation.

Counts tokens for the system message. System messages also contribute to the total token count and should be considered in prompt engineering.

In a web application like the one we are building, you can use the `countTokens` function to check if a message exceeds the token limit before sending it to the AI model. By keeping a current tally of available tokens, we can add an indicator next to the input text that shows the user how many tokens they've used as they type. Consider implementing this feature on your own as an extra exercise.

In Google's Gemini, which doesn't provide a standalone tokenizer like OpenAI's tiktoken, you can't count tokens locally yet. Instead, use the Google API itself, which returns token usage info as part of its response. This means you send your prompt to the API and check the token count from the result.